Virtualisation and Cloud Technologies

CEM YILMAZ
Faculty № 22572118
Technical University of Gabrovo
Faculty of
Electrical Engineering and Electronics

Project made for the degree of
Software and Computer Engineering
June 5, 2026

Contents

1 Introduction

Virtualisation is a method of isolating and abstracting computing resources, allowing multiple virtual instances to run on a single physical machine. A virtual machine is defined as an efficient, isolated duplicate of a real machine (Popek & Goldberg, 1974). Popek and Goldberg categorise three essential characteristics of a virtual machine: an "essentially identical" environment, efficiency, and resource control (Popek & Goldberg, 1974).

Virtualisation has become a fundamental technology in modern computing, enabling the efficient use of hardware resources, improving scalability, and facilitating the deployment of applications in cloud environments. However, at the heart of virtualisation and its technologies lies two distinct methods of achieving it: the type-1 (bare-metal) and type-2 (hosted) hypervisors (Susnjara & Smalley, 2024).

Type-1 hypervisors run directly on the host's physical hardware allowing it to directly interact with the underlying infrastructure such as the central processing unit (CPU), memory, graphical compute, and storage. The direct access to hardware resources allows type-1 hypervisors to provide better performance and scalability compared to type-2 hypervisors. Type-1 hypervisors are commonly used in enterprise environments and data centers where performance and security are critical. Examples of type-1 hypervisors include VMware ESXi, Microsoft Hyper-V, Citrix Hypervisor, Kernel-based Virtual Machine (KVM) and its derivatives e.g. Proxmox VE (Susnjara & Smalley, 2024).

Type-2 hypervisors, however, do not directly run in the underlying infrastructure. Instead, type-2 hypervisors run as a layer on top of the host's existing operating system (OS). For communication with the physical layer, the type-2 hypervisor creates a bridge between the OS and the virtualised environment. This additional layer of abstraction can lead to performance overhead compared to type-1 hypervisors, as it directly affects the Popek and Goldberg criterion of efficiency. Specifically, a statistically dominant subset of instructions must execute directly on the physical processor without VMM intervention (Popek & Goldberg, 1974). Type-2 hypervisors are often used for desktop virtualisation and testing environments where ease of use and flexibility are more important than raw performance. Examples of type-2 hypervisors include VMWare workstation, Oracle VirtualBox, Microsoft Virtual PC (Susnjara & Smalley, 2024).

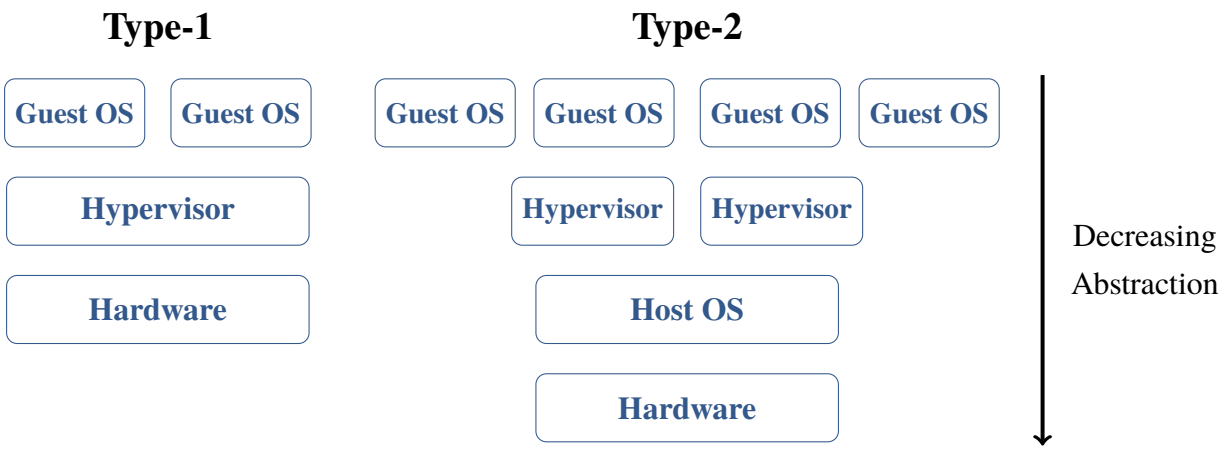

Figure 1: Architectural comparison of type-1 and type-2 hypervisors.

Modern virtualisation technologies have evolved to minimise overhead and improve performance for both types of hypervisors. For instance, hardware-assisted virtualisation features such as Intel VT-x and AMD-V have been introduced to enhance the performance of virtual machines by allowing certain instructions to be executed directly on the physical CPU, reducing the need for VMM intervention (Susnjara & Smalley, 2024). Additionally, advancements in memory management and I/O virtualisation have further improved the efficiency of virtualised environments (Barham et al., 2003). However, the question of whether measurable difference remain and under what workload conditions is the central concern of this paper.

This paper presents a comparative performance evaluation of type-1 and type-2 hypervisor technologies under controlled laboratory benchmarks. The methodology, results and analysis that follow aim to determine whether the theoretical performance advantage of type-1 hypervisors is still observable in practice today.

2 Methodology

2.1 Test Environment

All benchmarks were conducted on a single physical host machine to ensure consistency across both hypervisor configurations. The host hardware specifications are detailed in Table 1.

Table 1: Host machine hardware specifications.

Component	Specification
CPU	Intel Core i5-13600K (14 cores, 20 threads)
RAM	64 GB DDR4 @ 3200 MHz
Storage	Seagate Barracuda 2TB HDD (ST2000DM008, SATA III, 7200 RPM)
OS (Type 2 host)	Microsoft Windows 11 25H2 (Build 26200.8524)

2.2 Hypervisor Configuration

Two hypervisors were selected to represent each architectural type:

- **Type-1: Microsoft Hyper-V 10.0.26100.8457 (Windows 11 25H2, OS Build 26200.8524).** Hyper-V is a native Type-1 hypervisor integrated into Windows 11, running as a thin hypervisor layer beneath the host OS (Microsoft, 2025). It was enabled via Windows Features on the existing Windows 11 25H2 installation. Initial configuration used Generation 1, which enforces BIOS firmware and an emulated IDE disk controller. As IDE is an emulated device introducing additional I/O processing overhead unrelated to hypervisor architecture, the VM was recreated as Generation 2, which uses UEFI firmware and a synthetic small computer system interface (SCSI) controller.
- **Type-2: Oracle VirtualBox 7.2.8.** VirtualBox was installed on the Windows 11 host OS. As a hosted hypervisor, VirtualBox relies on Windows for hardware access, introducing an additional software layer between the guest VM and physical hardware. This is the architectural source of overhead being measured in this experiment. Accessed at https://download.virtualbox.org/virtualbox/7.2.8/VirtualBox-7.2.8-173730-Win.exe.

Prior to benchmarking for VirtualBox, Windows 11 Virtualization Based Security (VBS) and Memory Integrity were disabled, and the Hyper-V hypervisor launch type was set to off via `bcdedit /set hypervisorlaunchtype off`. This was necessary to ensure VirtualBox operated as a true Type-2 hypervisor with direct access to the host hardware, rather than as a guest of the Hyper-V hypervisor which Windows 11 enables by default. Without this step, VirtualBox would be running as a potential Hyper-V guest, invalidating the Type-1 vs Type-2 architectural comparison. Additionally, almost all non-essential background tasks were killed on the host OS to minimise interference with the benchmarks, and the host was rebooted after each hypervisor configuration change to ensure a clean state.

2.3 Virtual Machine Configuration

To isolate hypervisor architecture as the sole independent variable, both virtual machines were configured identically across all parameters where possible. The guest operating system was **Ubuntu Server 26.04 LTS** in both cases, installed from the same ISO image and accessed at https://releases.ubuntu.com/26.04/ubuntu-26.04-live-server-amd64.iso. The VM specifications are summarised in Table 2.

Table 2: Virtual machine configuration for both hypervisors.

Parameter	Value
vCPUs	4
RAM	8 GB
Disk size	50 GB
Disk allocation	Fixed size
Disk controller	SCSI
Firmware	UEFI
Guest OS	Ubuntu Server 26.04 LTS
Secure Boot	Disabled
Swap	Disabled

The disk was allocated as a fixed size image on both hypervisors rather than dynamically allocated, to eliminate allocation time overhead from influencing I/O benchmark results. SCSI was used as the disk controller on both VMs to ensure a consistent, stable controller baseline. VirtualBox uses an emulated LsiLogic SCSI controller (Corporation, 2026), while Hyper-V Generation 2 uses a fully synthetic SCSI controller via VMBus (Howard, 2013). Both were selected as the closest comparable storage controllers available on their respective hypervisors, to minimise architectural differences between the two configurations.

Swap was disabled on both VMs prior to benchmarking (`sudo swapoff -a`) to ensure that memory benchmark results reflect RAM throughput exclusively, without spilling to disk.

2.4 Benchmark Tools

Two common benchmarking tools were used:

- **sysbench**. Used for CPU and memory benchmarks. For the purpose of this experiment, the version used was 1.0.20.
- **fio**. Used for disk I/O benchmarks. For the purpose of this experiment, the version used was 3.41.

Both tools were installed from the Ubuntu package repositories on each VM independently.

2.5 Benchmark Procedures

Each benchmark was executed five times per VM, and the arithmetic mean of the five runs was taken as the reported result. Prior to each test session, the VM was booted and left idle for two minutes to allow background OS processes to settle. No other VMs or applications were running on the host during any test.

2.5.1 CPU Benchmark

The sysbench CPU benchmark was used to measure computational throughput. The test calculates prime numbers up to a configurable upper bound using all available vCPU threads, and reports the number of events completed per second.

```
sysbench cpu --cpu-max-prime=20000 --threads=4 --time=60 run
```

Listing 1: sysbench CPU benchmark command.

The `-threads=4` parameter matches the vCPU allocation of each VM, and the test duration was fixed at 60 seconds. The value of 20,000 was selected as it is a widely used upper bound in sysbench CPU benchmark.

2.5.2 Memory Benchmark

The sysbench memory benchmark was used to measure memory subsystem throughput. The test performs sequential write operations to memory using 1 MB blocks until a total transfer of 10 GB has been completed using 4 threads, reporting throughput in MiB/s.

```
sysbench memory --memory-block-size=1M --memory-total-size=10G --threads=4
    run
```

Listing 2: sysbench memory benchmark command.

2.5.3 Disk I/O Benchmark

Disk I/O performance was evaluated using fio with a commonly used value of 4K for both random read and random write workloads. The I/O engine used was libaio (Linux asynchronous I/O), with 4 parallel jobs matching the vCPU count.

A throwaway fio run was performed before each workload type to prepare the disk cache, ensuring that subsequent measurements reflect steady I/O performance rather than cold cache behaviour. The warmup run uses the same name as the benchmark it precedes so that fio reuses the pre-created files during the actual benchmark runs. After the random read warmup and five benchmark runs, the randread files were deleted before performing the random write warmup.

```
fio --name=randread --ioengine=libaio --rw=randread \
    --bs=4k --size=64M --numjobs=4 --runtime=60 \
    --direct=1 --group_reporting
```

Listing 3: fio random read benchmark command.

```
fio --name=randwrite --ioengine=libaio --rw=randwrite \
    --bs=4k --size=64M --numjobs=4 --runtime=60 \
    --direct=1 --group_reporting
```

Listing 4: fio random write benchmark command.

2.6 Controlled Variables

Table 3 summarises the variables held constant across both test environments to ensure the validity of the comparison.

Table 3: Controlled variables across both hypervisor configurations.

Variable	Control measure
Guest OS	Identical Ubuntu Server 26.04 LTS install
vCPU count	4 on both VMs
RAM allocation	8 GB on both VMs
Disk size	50 GB fixed allocation on both VMs
Disk controller	SCSI on both VMs
Firmware type	UEFI on both VMs
Swap	Disabled on both VMs
Concurrent load	No other VMs or applications running
Idle period	2 minutes post-boot before each test session
Benchmark commands	Identical commands on both VMs
Run count	5 runs per benchmark

3 Results

3.1 Visualisation Methodology

Each benchmark subsection presents results using a radar chart, which allows multiple performance metrics to be compared simultaneously on a single figure. Because some of the metrics can differ in unit and magnitude, all values are normalised to the range $[0, 1]$ before plotting.

Let m_i denote the arithmetic mean of metric i across the five benchmark runs, and let $R_i > 0$ be a predefined reference maximum for that metric. The normalised score $\hat{s}_i$ is given by

$$\hat{s}_i = \frac{m_i}{R_i}, \text{ where } \hat{s}_i \in [0, 1] \tag{1}$$

The reference maximum R_i for each metric is chosen as a conservative upper bound that strictly exceeds all observed values across both hypervisor configurations, that is

$$R_i = \kappa \cdot \max\left(m_i^{\text{VBox}},\ m_i^{\text{Hyper-V}}\right),\ \text{where } \kappa > 1 \tag{2}$$

where $\kappa > 1$ is chosen such that no data point saturates its axis, ensuring the full scale of the chart remains interpretable. Applying identical reference maxima to both hypervisors ensures that each axis represents the same absolute scale for both, making the two polygons directly comparable.

For metrics where a lower value indicates better performance (e.g. latency, standard deviation), the score is inverted so that a larger enclosed polygon area consistently represents the better performance. Namely,

$$\hat{s}_i^{\text{inv}} = 1 - \hat{s}_i = 1 - \frac{m_i}{R_i} \tag{3}$$

Equations (1)–(3) are applied uniformly across the CPU, memory, and disk I/O benchmark sections that follow.

3.2 CPU Benchmarks

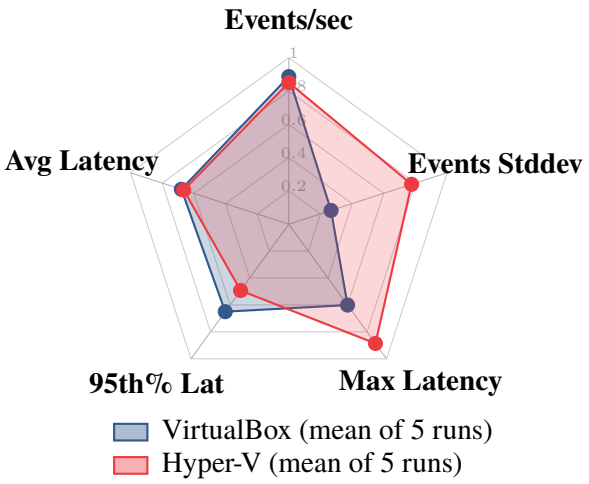

Figure 2: CPU benchmark performance radar chart.

Figure 2 shows the normalised performance profiles of both hypervisors across five chosen CPU metrics. Furthermore, the following reference maxima were chosen: events/sec $R = 7{,}000$, average latency $R = 2$ ms, 95th percentile latency $R = 2$ ms, maximum latency $R = 20$ ms, standard deviation $R = 700$. Latency and standard deviation axes are inverted per equation (3), such that a larger enclosed area indicates better performance.

Table 4: CPU benchmark mean comparison (VirtualBox vs Hyper-V).

Metric	VirtualBox (mean)	Hyper-V (mean)	Difference	Difference (%)
Events/sec	6206.11	5958.36	−247.75	−3.99%
Total Events	372375	357510	−14865	−3.99%
Min Latency (ms)	0.62	0.62	0.00	0.00%
Avg Latency (ms)	0.642	0.672	+0.030	+4.67%
Max Latency (ms)	7.98	2.29	−5.69	−71.30%
95th% Lat (ms)	0.700	1.014	+0.314	+44.86%
Events Stddev	513.20	156.65	−356.55	−69.47%

Differences are reported as Hyper-V minus VirtualBox. A negative value indicates VirtualBox is higher, a positive value indicates Hyper-V is higher. For latency and standard deviation metrics, lower values are better.

3.3 Memory Benchmarks

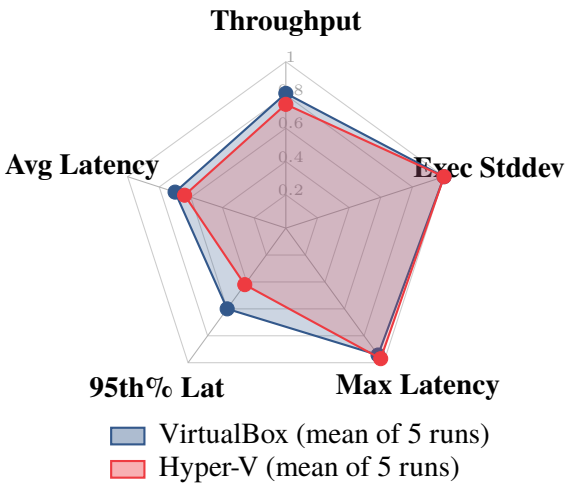

Figure 3: Memory benchmark performance radar chart (VirtualBox vs Hyper-V).

Figure 3 shows the normalised performance profiles of both hypervisors across five chosen memory metrics. Furthermore, the following reference maxima were chosen: throughput $R = 140{,}000\,\text{MiB/s}$, average latency $R = 0.10\,\text{ms}$, 95th percentile latency $R = 0.10\,\text{ms}$, maximum latency $R = 15\,\text{ms}$, execution standard deviation $R = 0.05$. Latency and standard deviation axes are inverted per equation (3), such that a larger enclosed area indicates better performance.

Table 5: Memory benchmark mean comparison (VirtualBox vs Hyper-V).

Metric	VirtualBox (mean)	Hyper-V (mean)	Difference	Difference (%)
Throughput (MiB/s)	113389.76	104110.83	−9278.93	−8.18%
Min Latency (ms)	0.020	0.020	0.000	0.00%
Avg Latency (ms)	0.030	0.036	+0.006	+20.00%
Max Latency (ms)	0.860	0.450	−0.410	−47.67%
95th% Lat (ms)	0.040	0.058	+0.018	+45.00%

Differences are reported as Hyper-V minus VirtualBox. A negative value indicates VirtualBox is higher, a positive value indicates Hyper-V is higher. For latency metrics, lower values are better.

3.4 Disk I/O Benchmarks

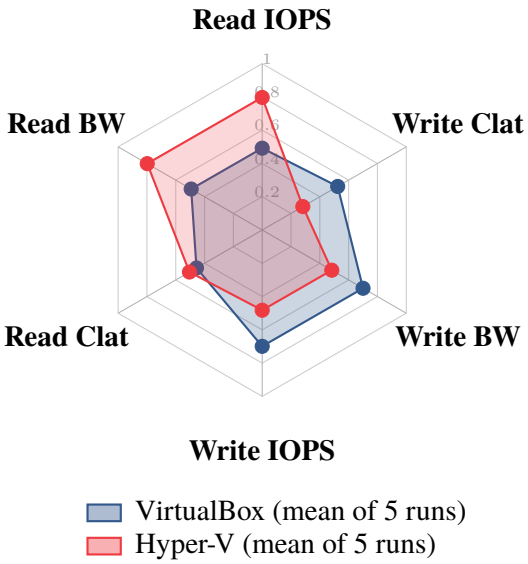

Figure 4: Disk I/O benchmark performance radar chart (VirtualBox vs Hyper-V).

Figure 4 shows the normalised performance profiles of both hypervisors across six chosen disk I/O metrics covering both random read and random write workloads. Furthermore, the following reference maxima were chosen: read IOPS $R = 1{,}000$, read bandwidth $R = 4{,}000$ KiB/s, read completion latency $R = 15{,}000\,\mu$s, write IOPS $R = 300$, write bandwidth $R = 1{,}200$ KiB/s, write completion latency $R = 40{,}000\,\mu$s. Completion latency axes are inverted per equation (3), such that a larger enclosed area indicates better performance.

Table 6: Disk I/O random read benchmark mean comparison (VirtualBox vs Hyper-V).

Metric	VirtualBox (mean)	Hyper-V (mean)	Difference	Difference (%)
IOPS	492.2	797.0	+304.8	+61.93%
BW (KiB/s)	1970.8	3190.0	+1219.2	+61.86%
Avg BW (KiB/s)	1975.95	4606.13	+2630.18	+133.11%
Avg IOPS	492.58	1151.52	+658.94	+133.79%
BW Stdev (KiB/s)	143.08	1110.80	+967.72	+676.35%
IOPS Stdev	35.81	277.68	+241.87	+675.26%
Avg Clat (μs)	8158.80	7425.41	−733.39	−8.99%
Clat Stdev (μs)	15333.17	21167.46	+5834.29	+38.05%

Table 7: Disk I/O random write benchmark mean comparison (VirtualBox vs Hyper-V).

Metric	VirtualBox (mean)	Hyper-V (mean)	Difference	Difference (%)
IOPS	209.6	144.6	−65.0	−31.01%
BW (KiB/s)	840.2	580.0	−260.2	−30.97%
Avg BW (KiB/s)	6275.11	671.36	−5603.75	−89.30%
Avg IOPS	1567.51	167.83	−1399.68	−89.29%
BW Stdev (KiB/s)	1162.47	142.33	−1020.14	−87.76%
IOPS Stdev	290.66	35.58	−255.08	−87.76%
Avg Clat (μs)	19057.61	28701.77	+9644.16	+50.61%
Clat Stdev (μs)	286258.54	95158.01	−191100.53	−66.76%

Differences are reported as Hyper-V minus VirtualBox. A negative value indicates VirtualBox is higher, a positive value indicates Hyper-V is higher. For completion latency and standard deviation metrics, lower values are better.

4 Discussion

4.1 CPU Performance

VirtualBox achieved a slightly higher mean throughput of 6,206 events/sec compared to Hyper-V's 5,958 events/sec, a difference of roughly 4%. Hyper-V, however, produced considerably more consistent results: its maximum observed latency was 2.29 ms versus VirtualBox's 7.98 ms, and its standard deviation of events across runs was nearly 70% lower. In other words, VirtualBox was marginally faster on average, but Hyper-V behaved more predictably every run.

This pattern is consistent with findings in the literature. Giallorenzo et al., benchmarking CPU performance across multiple virtualisation technologies using Dhrystone and Whetstone have

found that type-2 hypervisors performed within a few percent of bare-metal for both integer and floating-point workloads, with no category decisively outperforming the other (Giallorenzo et al., 2021). The narrow throughput gap observed here follows the same pattern. Under purely arithmetic workloads, hardware-assisted virtualisation allows the guest to execute calculations directly on the physical processor, meaning the additional software layer beneath a type-2 hypervisor makes little practical difference to throughput. The greater run consistency observed in Hyper-V, hovewer, aligns with a known benefit of type-1 architectures, where the guest is not competing with a host OS for CPU scheduling time.

4.2 Memory Performance

VirtualBox recorded a mean memory throughput of 113,390 MiB/s, an 8.2% advantage over Hyper-V's 104,111 MiB/s. Hyper-V showed slightly higher average and 95th percentile latency, though it achieved a lower maximum latency. Across the memory metrics, VirtualBox consistently came out marginally ahead.

This result may seem surprising given that Hyper-V is a type-1 hypervisor, but prior work shows that memory performance does not follow a simple type-1 versus type-2 pattern. Giallorenzo et al. tested VMs using the RAMspeed tool across six virtualisation technologies and has found that memory performance was largely unaffected by virtualisation overhead regardless of hypervisor type, with most technologies matching or slightly exceeding bare-metal and only VirtualBox showing a modest drop of around 5% (Giallorenzo et al., 2021). Their results suggest that memory throughput is influenced more by how a given implementation manages address translation and cache access than by architectural category alone (Giallorenzo et al., 2021). A plausible explanation in our case is that VirtualBox, running on top of Windows 11, benefits from memory already mapped by the host OS, reducing some address translation overhead for bulk sequential memory transfers within the virtual machine.

4.3 Disk I/O Performance

The disk results showed a clear split between read and write workloads. For random reads, Hyper-V substantially outperformed VirtualBox, delivering 797 IOPS compared to 492 IOPS, an advantage of 62%. For random writes, VirtualBox came out ahead with 210 IOPS against Hyper-V's 145 IOPS, with an advantage of 31%. Hyper-V also had noticeably worse average write completion latency of 28,702 μs compared to 19,058 μs on the VirtualBox.

The read advantage for Hyper-V aligns with what other researchers have found. Giallorenzo et al. compared multiple virtualisation technologies using Bonnie++ and DBENCH, finding that type-1 hypervisors generally delivered more consistent block I/O read performance, attributed to a more direct path to the hardware (Giallorenzo et al., 2021). Under VirtualBox, every I/O

request must pass through the Windows 11 host OS storage stack before reaching the disk, introducing additional overhead that limits read throughput.

The write reversal is unintuitive. Giallorenzo et al. observed similarly non-linear write behaviour in their results, noting that VirtualBox outperformed VMWare Workstation on several write benchmark results despite both being type-2 hypervisors. They attributed this to write buffering differences in the host OS storage stack (Giallorenzo et al., 2021). A similar mechanism plausibly explains the write reversal observed here. Namely, the Windows 11 storage stack buffers short burst writes before committing them to disk, improving apparent write throughput, while Hyper-V's more direct disk access enforces stricter write ordering, increasing per write latency.

5 Conclusion

The results demonstrate that the theoretical performance advantage of type-1 hypervisors over type-2 is not uniformly observable in practice. VirtualBox marginally outperformed Hyper-V across CPU and memory metrics, suggesting that the additional software layer of a type-2 hypervisor introduces little overhead for these workloads. The most meaningful difference between the two was observed in disk I/O. Hyper-V delivered substantially better random read performance, while VirtualBox delivered substantially better random write performance. This read/write split reflects the architectural difference between the two hypervisors more clearly than CPU or memory results alone.

Additionally, our findings are broadly in line with the benchmarking literature. Giallorenzo et al. similarly found that differences between virtualisation categories were workload dependent, with no single architecture dominating across all resource types, and that results could be especially counterintuitive (Giallorenzo et al., 2021). In particular, memory and disk I/O performance were shown to be influenced by host buffering and storage stack behaviour, introducing additional factors independent of hypervisor architecture.

The practical implication is that the choice between a type-1 and type-2 hypervisor should be guided by the specific workload profile of the intended application. For read intensive or latency sensitive workloads, Hyper-V's more direct hardware access offers a measurable advantage. For general purpose or memory bound workloads, the overhead introduced by VirtualBox's additional software layer is less significant than the architectural difference between the two hypervisors. This narrowing of the performance gap is likely a product of decades of optimisation using techniques such as nested paging and KVM paravirtualisation.
The formal requirements laid out by Popek and Goldberg (Popek & Goldberg, 1974) in 1974 described a world where virtualisation was achieved largely in software, and the overhead was

correspondingly significant. The subsequent introduction of hardware assisted virtualisation features such as Intel VT-x and AMD-V effectively collapsed much of the architectural distinction between type-1 and type-2 hypervisors at the instruction execution level. Where the performance gap between these two architectural approaches would have been clearly visible in the 1970s and 1980s, modern hardware optimisations have reduced it to the point where for most workloads it is unlikely to be the deciding factor.

References

Barham, P., Dragovic, B., Fraser, K., Hand, S., Harris, T., Ho, A., Neugebauer, R., Pratt, I., & Warfield, A. (2003). Xen and the art of virtualization. *37*, 164–177. https://doi.org/10.1145/1165389.945462

Corporation, O. (2026, February). *Chapter 5. Virtual Storage*. Retrieved June 5, 2026, from https://www.virtualbox.org/manual/ch05.html

Giallorenzo, S., Mauro, J., Poulsen, M. G., & Siroky, F. (2021). Virtualization Costs: Benchmarking Containers and Virtual Machines Against Bare-Metal. *SN Computer Science*, *2*(5), 404. https://doi.org/10.1007/s42979-021-00781-8

Howard, J. (2013). *Hyper-V generation 2 virtual machines part 3*. Retrieved June 5, 2026, from https://learn.microsoft.com/en-us/archive/blogs/jhoward/hyper-v-generation-2-virtual-machines-part-3

Microsoft. (2025, August 5). *Hyper-V virtualization in Windows Server and Windows*. Retrieved June 4, 2026, from https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/overview

Popek, G. J., & Goldberg, R. P. (1974). Formal requirements for virtualizable third generation architectures. *Communications of the ACM*, *17*(7), 412–421. https://doi.org/10.1145/361011.361073

Susnjara, S., & Smalley, I. (2024, October 30). *What Are Hypervisors? | IBM*. Retrieved June 3, 2026, from https://www.ibm.com/think/topics/hypervisors

Appendices

A CPU Benchmark Results

Table 8: VirtualBox CPU benchmark results (sysbench, 5 runs).

Run	Events/sec	Total Events	Min Lat (ms)	Avg Lat (ms)	Max Lat (ms)	95th% (ms)	Events Stddev
1	6222.59	373364	0.62	0.64	8.95	0.69	512.34
2	6124.09	367454	0.62	0.65	12.19	0.74	456.60
3	6225.11	373516	0.62	0.64	6.70	0.69	525.66
4	6231.81	373917	0.62	0.64	6.44	0.69	543.40
5	6226.93	373624	0.62	0.64	5.60	0.69	527.98
Mean	**6206.11**	**372375**	**0.62**	**0.642**	**7.98**	**0.700**	**513.20**

Table 9: Hyper-V CPU benchmark results (sysbench, 5 runs).

Run	Events/sec	Total Events	Min Lat (ms)	Avg Lat (ms)	Max Lat (ms)	95th% (ms)	Events Stddev
1	5977.82	358678	0.62	0.67	1.75	0.90	218.28
2	5981.75	358915	0.62	0.67	3.70	0.89	173.51
3	5974.32	358468	0.62	0.67	2.40	0.97	86.89
4	5946.04	356771	0.62	0.67	1.87	1.10	62.86
5	5911.87	354720	0.62	0.68	1.73	1.21	241.70
Mean	**5958.36**	**357510**	**0.62**	**0.672**	**2.29**	**1.014**	**156.65**

B Memory Benchmark Results

Table 10: VirtualBox memory benchmark results (sysbench, 5 runs).

Run	Throughput (MiB/s)	Total Ops	Min Lat (ms)	Avg Lat (ms)	Max Lat (ms)	95th% (ms)	Exec Stddev
1	111445.64	10240	0.02	0.03	1.57	0.04	0.00
2	112116.21	10240	0.02	0.03	0.92	0.04	0.00
3	116511.03	10240	0.02	0.03	0.79	0.04	0.00
4	114039.31	10240	0.02	0.03	0.37	0.04	0.00
5	112836.60	10240	0.02	0.03	0.66	0.04	0.00
Mean	**113389.76**	**10240**	**0.02**	**0.030**	**0.86**	**0.040**	**0.000**

Table 11: Hyper-V memory benchmark results (sysbench, 5 runs).

Run	Throughput (MiB/s)	Total Ops	Min Lat (ms)	Avg Lat (ms)	Max Lat (ms)	95th% (ms)	Exec Stddev
1	120347.52	10240	0.02	0.03	1.20	0.04	0.00
2	90520.75	10240	0.02	0.04	0.27	0.07	0.00
3	90520.75	10240	0.02	0.04	0.27	0.07	0.00
4	101264.21	10240	0.02	0.04	0.25	0.07	0.00
5	117900.92	10240	0.02	0.03	0.24	0.04	0.00
Mean	**104110.83**	**10240**	**0.02**	**0.036**	**0.45**	**0.058**	**0.000**

C Disk I/O Benchmark Results

Table 12: VirtualBox random read disk I/O benchmark results (fio, 5 runs).

Run	IOPS	BW (KiB/s)	Avg BW (KiB/s)	Avg IOPS	BW Stdev (KiB/s)	IOPS Stdev	Avg Clat (μs)	Clat Stdev (μs)
1	450	1801	1803.18	449.37	117.83	29.50	8871.05	16321.64
2	453	1815	1819.68	453.39	119.52	29.91	8800.88	15522.85
3	482	1930	1936.37	482.84	138.08	34.56	8278.77	15305.16
4	549	2198	2203.06	549.24	170.78	42.75	7269.45	14422.80
5	527	2110	2117.44	528.08	169.18	42.34	7573.83	15093.38
Mean	**492.2**	**1970.8**	**1975.95**	**492.58**	**143.08**	**35.81**	**8158.80**	**15333.17**

Table 13: Hyper-V random read disk I/O benchmark results (fio, 5 runs).

Run	IOPS	BW (KiB/s)	Avg BW (KiB/s)	Avg IOPS	BW Stdev (KiB/s)	IOPS Stdev	Avg Clat (μs)	Clat Stdev (μs)
1	1208	4836	6656.42	1664.10	1966.31	491.50	2641.23	17627.05
2	784	3136	3586.73	896.61	1071.00	267.74	4699.18	17736.13
3	1407	5629	10425.61	2606.40	2100.86	525.22	2120.93	10220.93
4	331	1327	1337.34	334.34	334.55	83.64	12037.14	34537.07
5	255	1022	1024.56	256.14	81.30	20.32	15628.57	25716.13
Mean	**797.00**	**3190.00**	**4606.13**	**1151.52**	**1110.80**	**277.68**	**7425.41**	**21167.46**

Table 14: VirtualBox random write disk I/O benchmark results (fio, 5 runs).

Run	IOPS	BW (KiB/s)	Avg BW (KiB/s)	Avg IOPS	BW Stdev (KiB/s)	IOPS Stdev	Avg Clat (μs)	Clat Stdev (μs)
1	210	843	6626.44	1655.44	848.07	212.12	18960.08	301721.68
2	213	854	6773.45	1692.30	1737.64	434.43	18730.50	285416.99
3	195	782	6412.31	1601.81	877.75	219.42	20459.91	318313.24
4	213	852	5208.01	1300.53	1261.67	315.49	18754.88	247211.03
5	217	870	6355.35	1587.47	1087.22	271.86	18382.67	278629.77
Mean	**209.6**	**840.2**	**6275.11**	**1567.51**	**1162.47**	**290.66**	**19057.61**	**286258.54**

Table 15: Hyper-V random write disk I/O benchmark results (fio, 5 runs).

Run	IOPS	BW (KiB/s)	Avg BW (KiB/s)	Avg IOPS	BW Stdev (KiB/s)	IOPS Stdev	Avg Clat (μs)	Clat Stdev (μs)
1	92	369	435.97	108.99	130.33	32.58	42722.66	120067.75
2	159	638	735.81	183.95	148.37	37.09	24831.00	89098.41
3	158	634	716.83	179.21	150.17	37.54	25149.49	86951.14
4	170	680	779.86	194.93	147.21	36.80	23436.48	82198.34
5	144	579	688.32	172.08	135.59	33.90	27369.24	97474.40
Mean	**144.60**	**580.00**	**671.36**	**167.83**	**142.33**	**35.58**	**28701.77**	**95158.01**